

PULS EVENTS

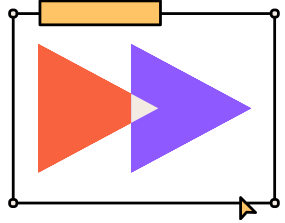
2026

POC CHATBOT INTELLIGENT



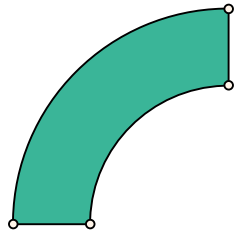
SOMMAIRE

Objectif et contexte du projet	1
Architecture technique	2
Données et modèle	3
Résultats et évaluation	4
Perspectives d'améliorations	5



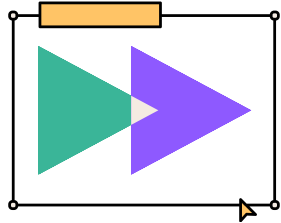
1. OBJECTIFS ET CONTEXTE

Puls-Events, entreprise technologique spécialisée dans les recommandations culturelles personnalisées, souhaite enrichir sa plateforme avec un nouveau chatbot conversationnel. L'objectif de cette initiative est d'offrir aux utilisateurs une interface capable de répondre de manière précise et naturelle à leurs questions sur les événements culturels à venir. Pour y parvenir, le projet s'appuie sur la conception d'une architecture RAG (Retrieval-Augmented Generation), une approche qui combine la puissance de la recherche vectorielle pour extraire les informations factuelles et les capacités des modèles de langage (LLM) pour la formulation des réponses.



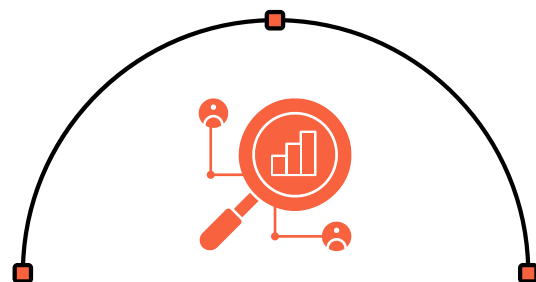
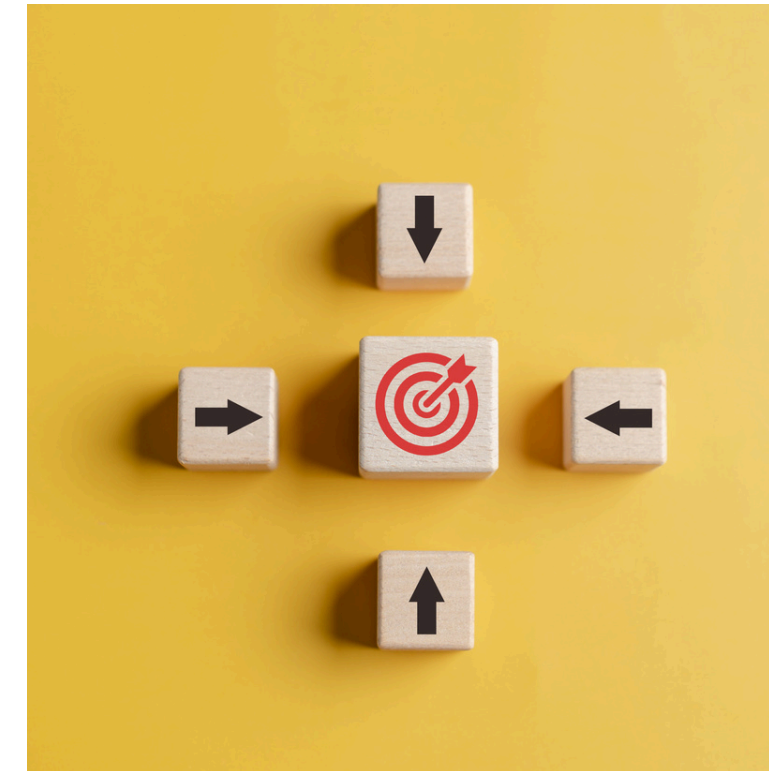
La mission consiste à concevoir, évaluer et livrer un Proof of Concept (POC) complet de ce système d'intelligence artificielle. Ce livrable technique prendra la forme d'une API, directement exploitable par les équipes produit et marketing pour leurs futurs développements. L'enjeu principal de cette démonstration est de valider la viabilité de la solution et prouver sa faisabilité technique.





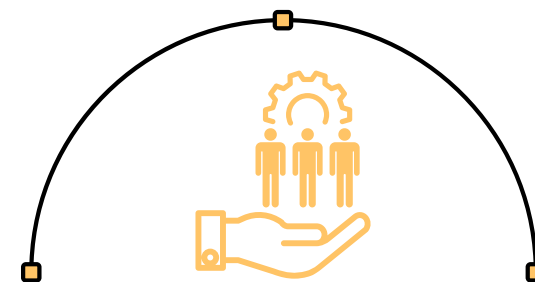
2. ARCHITECTURE TECHNIQUE

Le système suit un pipeline RAG classique :
Ingestion → vectorisation → indexation FAISS → récupération
sémantique → génération augmentée par contexte via un LLM.



Téléchargement des données

Téléchargement des données sur le site
opendatasoft au format json.
Données de l'année en cours
(Jan. → Déc. 2026)



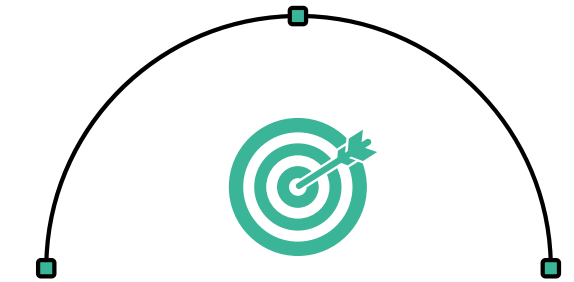
Création de l'index FAISS

Création de la base vectorielle à l'aide
de gemini-embedding-001 à partir des
données téléchargées.
Sauvegarde locale de l'index et des
chunks.



Rag + LLM

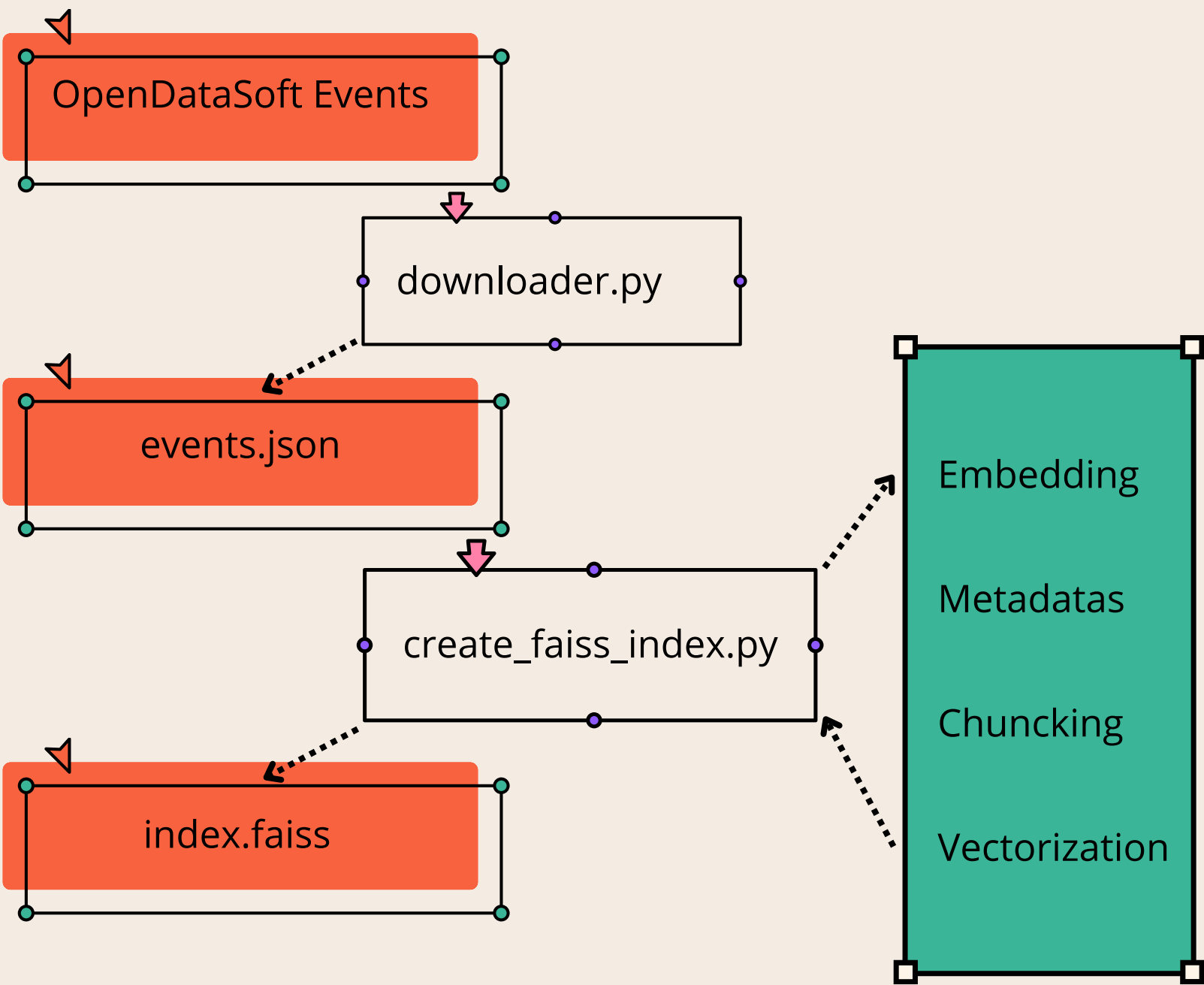
User Prompt + recherche Rag +
System Prompt.
LLM Gemini flash 2.5 compromis
rapidité/efficacité/cout.



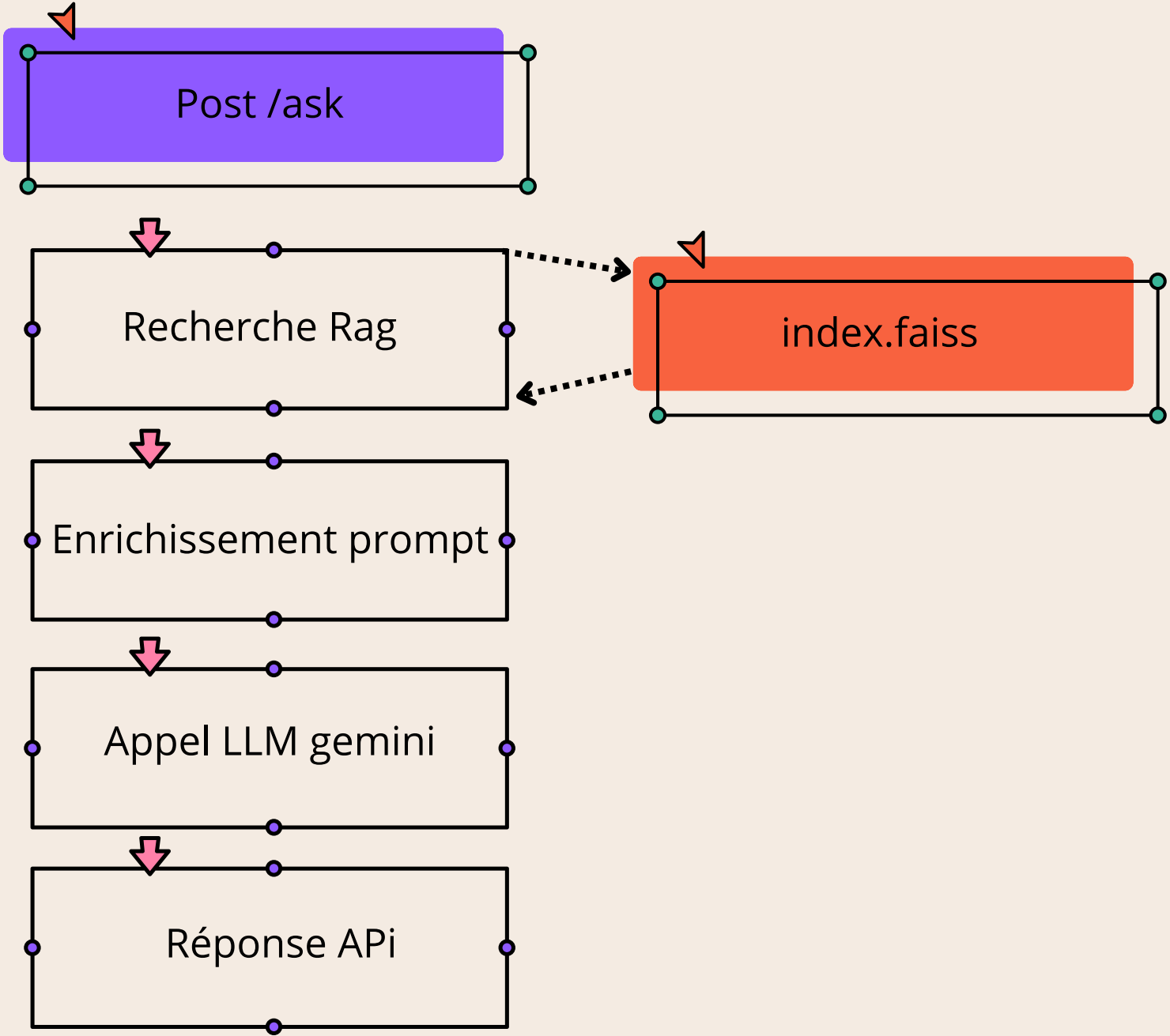
API FastAPI

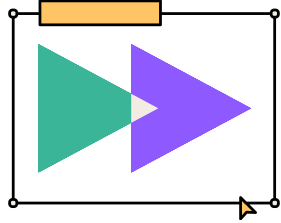
Endpoint API /ask et /rebuild pour
gérer l'index ainsi que les requêtes
utilisateurs.

BASE VECTORIELLE



API





3. DONNÉE ET MODÈLE 1/2

EXTRACTION ET CONCATENATION

Extraction et concaténation des champs pertinents :
titre, description, description longue, conditions, mots-clés, accessibilité, ville, département

NETTOYAGE

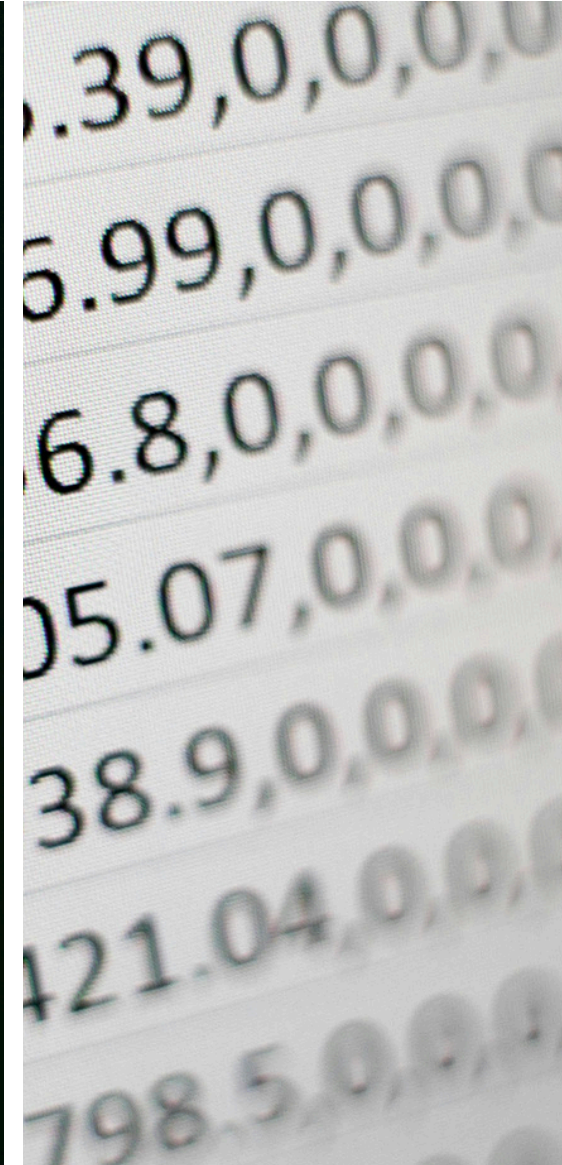
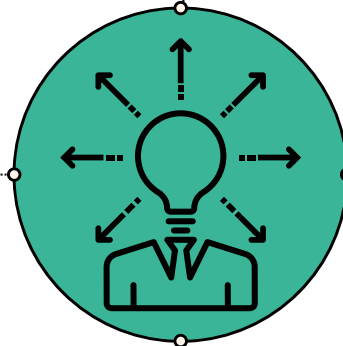
Nettoyage HTML des champs longs

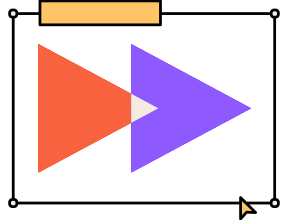
FORMATAGE

Formatage des dates d'ouverture (utils.format_dates) et des modalités d'inscription (utils.format_registration) en métadonnées

DECOUPAGE

Découpage en chunks via RecursiveCharacterTextSplitter (taille 1000, overlap 150) ;





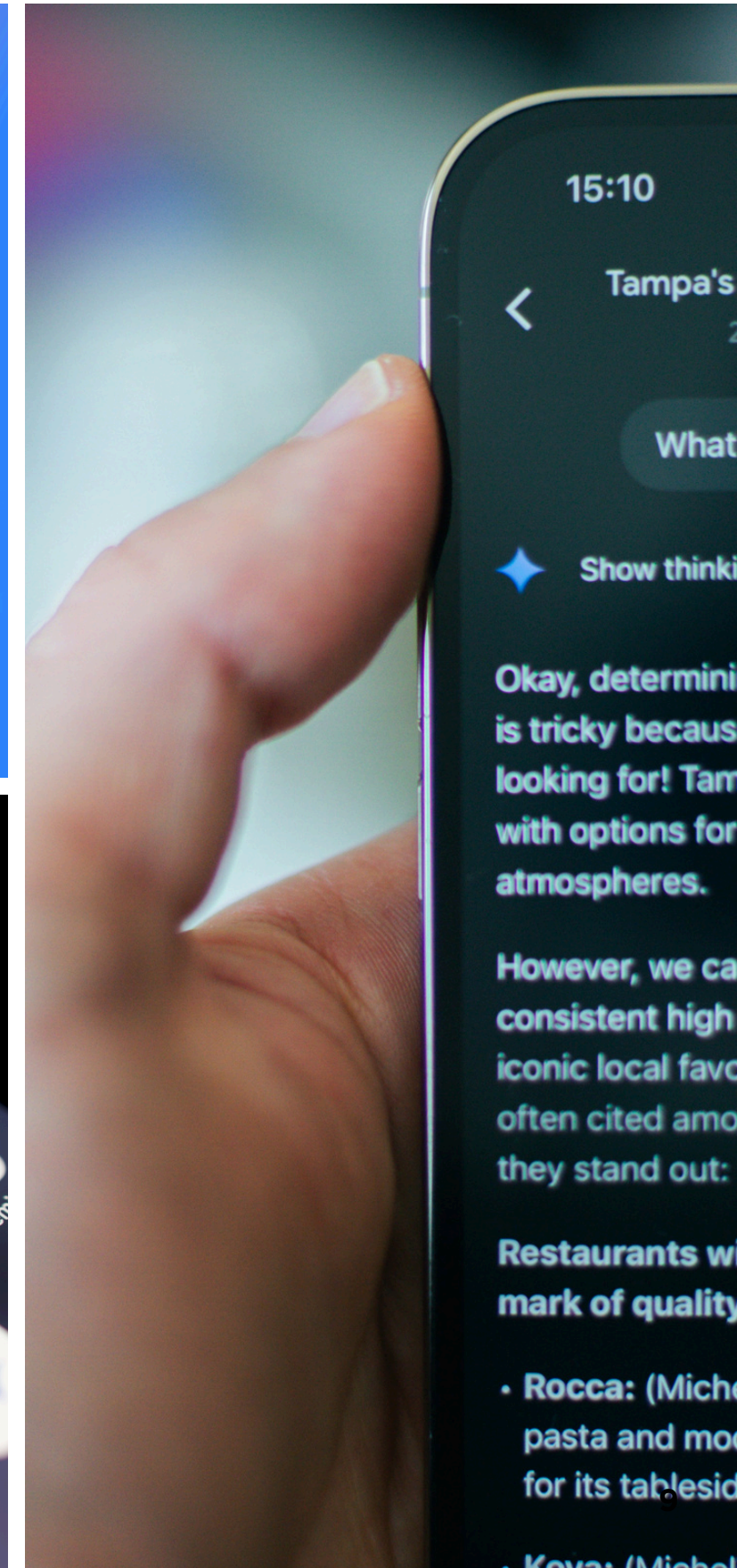
3. DONNÉES ET MODÈLE 2/2

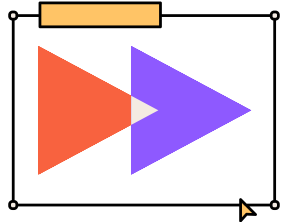
Embedding

Gemini Embedding est un moteur spécialisé dans la représentation vectorielle de grande dimension. Le modèle Gemini Embedding est idéal pour la recherche sémantique, la récupération de documents qui nécessitent des calculs de similarité rapides sur de grands ensembles de données.

LLM

Gemini 3.5 Flash-Lite est un modèle multimodal économique à faible latence, optimisé pour une exécution à haut débit et à faible coût pour les tâches de sous-agent et l'analyse de documents.





4. RÉSULTATS ET EVALUATION

Dashboard Langsmith

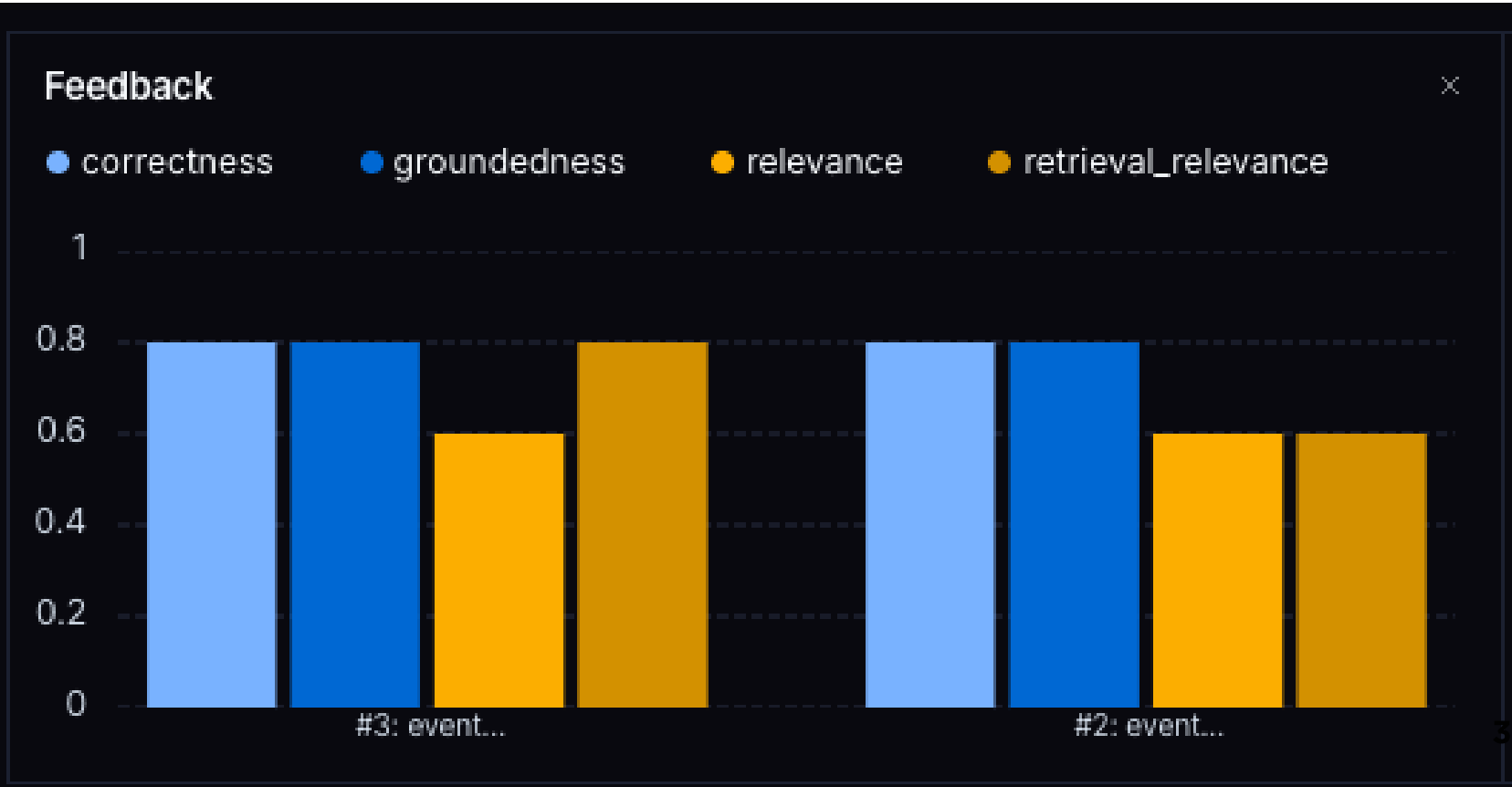
Résultats globaux

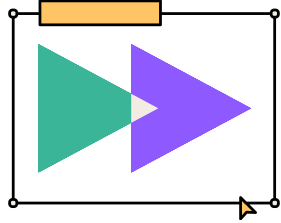
Le système RAG démontre de solides performances globales, affichant des scores moyens de 0.80 en exactitude, fidélité et qualité de recherche. S’il excelle sur les questions factuelles directes et gère correctement les requêtes hors périmètre en refusant de répondre, il peine en revanche sur les raisonnements temporels (comme identifier un "prochain" événement), ce qui entraîne des inexactitudes. De plus, la baisse du score moyen de pertinence (0.60) souligne principalement un biais dans le prompt d’évaluation actuel, qui pénalise à tort les refus justifiés du modèle plutôt qu’une réelle incapacité à adresser la question.

Métriques dévaluation choisies

- Correctness (Exactitude) : 0.80 en moyenne.
- Groundedness (Fidélité aux documents) : 0.80 en moyenne.
- Retrieval Relevance (Qualité de la recherche) : 0.80 en moyenne.
- Relevance (Pertinence de la réponse) : 0.60 en moyenne.

correctness ↓↑ 0.80 AVG	groundedness ↓↑ 0.80 AVG	relevance ↓↑ 0.60 AVG	retrieval_relevance ↓↑ 0.80 AVG
1.00	1.00	0.00	0.00
1.00	1.00	1.00	1.00
1.00	1.00	1.00	1.00
0.00	0.00	1.00	1.00
1.00	1.00	0.00	1.00





5. PERSPECTIVES D'AMÉLIORATIONS

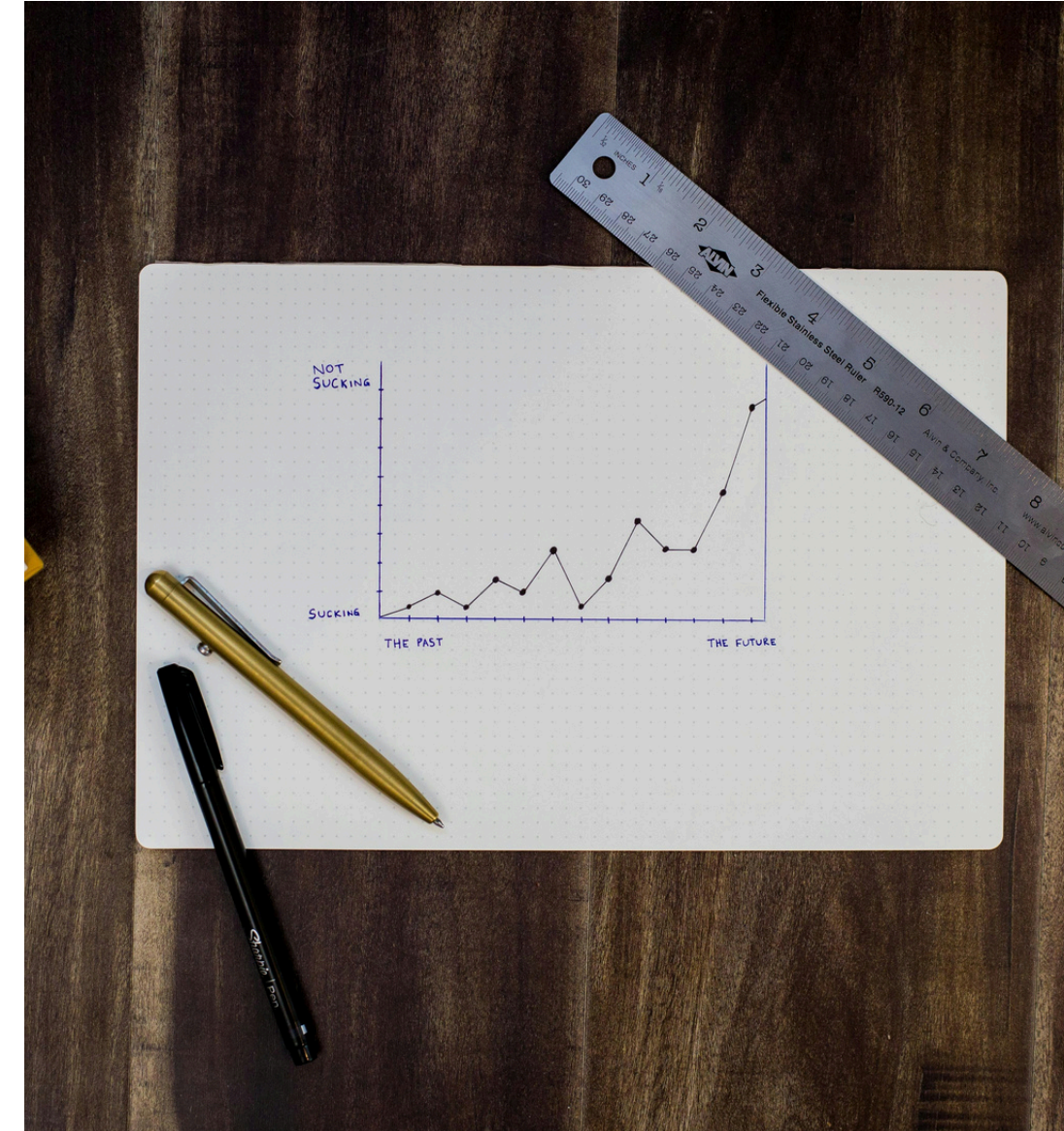
Injection du contexte temporel

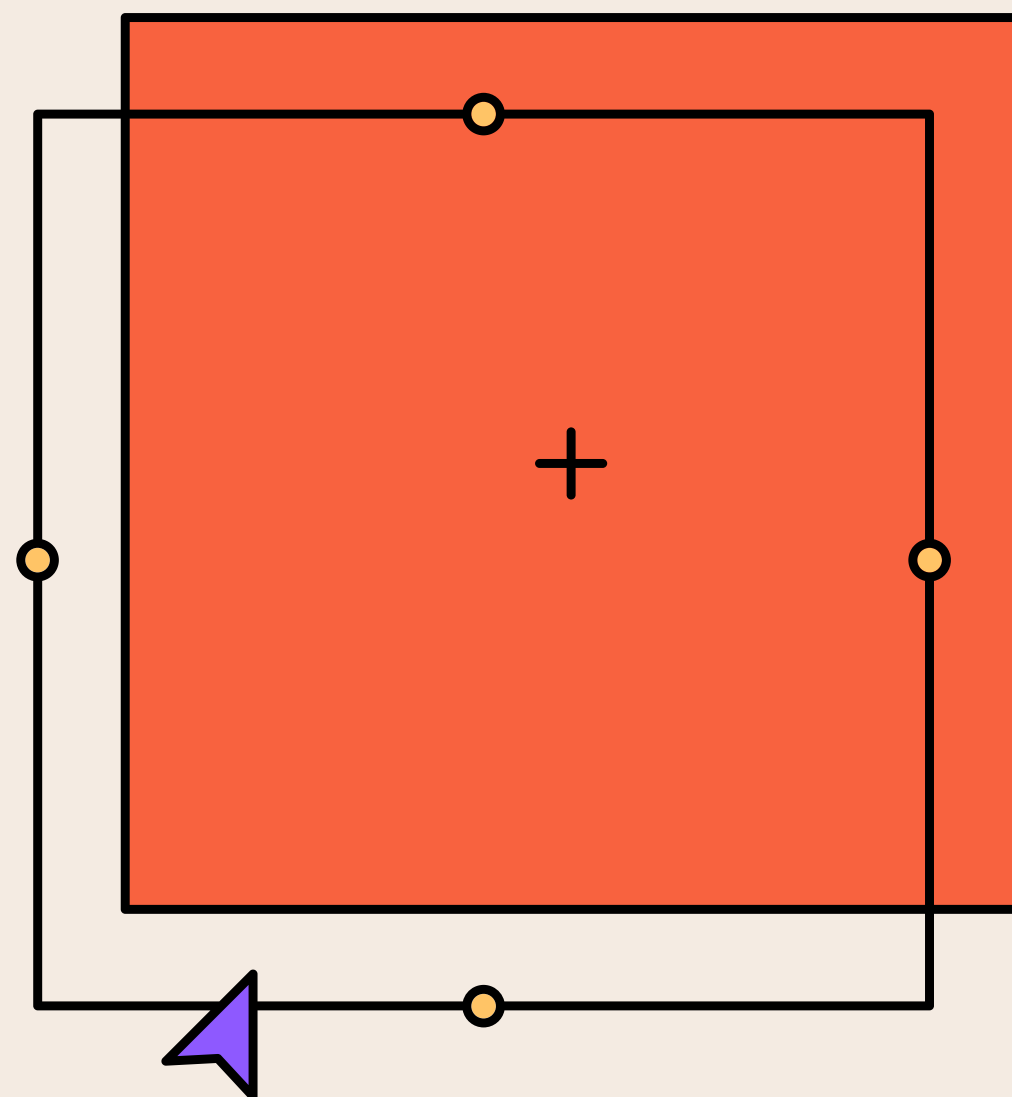
Actuellement, le system_prompt est statique. Il pourrait être intéressant d'injecter la date du jour dynamiquement lors de la construction du prompt, pour donner un contexte temporel au LLM.

Filtrage en amont

Si l'utilisateur demande le festival d'Angoulême, FAISS va quand même remonter les 5 documents les "moins éloignés", ce qui consomme des tokens et risque d'induire le LLM en erreur.

Utiliser un petit appel LLM très rapide pour extraire les entités de la question (ex: {"ville": "Angoulême", "année": 2027}), puis de passer ces valeurs comme filtres de métadonnées





MERCI !

DES QUESTIONS ?